Navigation and Routing in Flutter

Navigation is the process of moving between different screens (pages) in an application. In Flutter, screens are called **routes**, and navigation is handled by the **Navigator** class.

The Navigator manages a **stack of routes**. When a new screen is opened, it is **pushed onto the stack**, and when the user goes back, the route is **popped from the stack**.

Example stack:

```
None
Home Screen
     ↓
Profile Screen
     ↓
Settings Screen
```

If Settings is popped, the user returns to Profile.

---

1. Understanding App Structure and Navigation

Theory

In Flutter, applications are typically structured using **Widgets**. Each screen is usually implemented as a **StatelessWidget** or **StatefulWidget**.

Navigation allows users to move between these screens.

Typical Flutter App Structure:

```
None
lib/
 ├── main.dart
 ├── screens/
 │     ├── home_screen.dart
```

```
│     ├── detail_screen.dart
│     └── profile_screen.dart
└── routes.dart
```

Flutter navigation works using:

- **Navigator**
- **Route**
- **MaterialPageRoute**

Key idea:

```
None
Navigator.push() → Open new screen
Navigator.pop() → Go back
```

Example: Basic Navigation

```
None
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => SecondScreen(),
  ),
);
```

This pushes `SecondScreen` onto the navigation stack.

---

2. Navigation using push(), pushNamed(), and pop()

2.1 Navigator.push()

Theory

`Navigator.push()` is used to navigate to another screen by **directly providing the widget**.

It is suitable for **small applications**.

Example

```
None
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => SecondScreen(),
  ),
);
```

Full Example

```
None
ElevatedButton(
  child: Text("Go to Second Screen"),
  onPressed: () {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) => SecondScreen(),
      ),
    );
  },
);
```

2.2 Navigator.pop()

Theory

`Navigator.pop()` removes the current route from the stack and returns to the previous screen.

Example

```
Navigator.pop(context);
```

Example with Button

```
ElevatedButton(
  child: Text("Go Back"),
  onPressed: () {
    Navigator.pop(context);
  },
);
```

2.3 Navigator.pushNamed()

Theory

`pushNamed()` navigates using **route names instead of widgets**.

This makes navigation **cleaner and easier to manage in large applications**.

Example route:

```
/home
/profile
/settings
```

Example

Define routes in `MaterialApp`.

```
None
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => HomeScreen(),
    '/profile': (context) => ProfileScreen(),
  },
)
```

Navigate using:

```
None
Navigator.pushNamed(context, '/profile');
```

---

3. Passing Data Between Screens

Theory

Often, one screen needs to send data to another screen.

Example:

- Product list → Product details
- User list → User profile

Data can be passed using:

1. **Constructor parameters**
2. **Route arguments**

Method 1: Passing Data via Constructor

Example

```
None
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => DetailScreen(name: "John"),
  ),
);
```

Receiving Data

```
None
class DetailScreen extends StatelessWidget {
  final String name;

  DetailScreen({required this.name});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text("Detail")),
      body: Center(
        child: Text("Hello $name"),
      ),
    );
  }
}
```

---

4. Named Routing and Route Arguments

Theory

Named routes allow passing **structured arguments** between screens.

This approach is recommended for **larger applications**.

Passing Arguments

```
None
Navigator.pushNamed(
  context,
  '/detail',
  arguments: "Flutter Student",
);
```

Receiving Arguments

```
None
final args = ModalRoute.of(context)!.settings.arguments as
String;
```

Example usage:

```
None
Text("Welcome $args");
```

---

5. Best Practices in Route Management

To build **maintainable Flutter apps**, proper route management is important.

---

1. Use Named Routes in Large Apps

Bad:

```
None
Navigator.push(MaterialPageRoute...)
```

Good:

```
None
Navigator.pushNamed('/profile')
```

---

2. Centralize Routes

Create a separate file:

```
None
routes.dart
```

Example:

```
None
class AppRoutes {
  static const home = '/';
  static const profile = '/profile';
  static const detail = '/detail';
}
```

---

3. Avoid Deep Navigation Logic in UI

Use helper functions:

```
None
void goToProfile(BuildContext context) {
  Navigator.pushNamed(context, '/profile');
}
```

---

4. Use Meaningful Route Names

Bad:

```
None
'/s1'
```

Good:

```
None
'/userProfile'
```

Full Flutter Example

This example includes:

✔ push navigation
✔ named routes
✔ passing data
✔ route arguments
✔ pop navigation

Complete Flutter Code

```
None
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Navigation Demo',
```

```
      initialRoute: '/',

      routes: {
        '/': (context) => HomeScreen(),
        '/second': (context) => SecondScreen(),
        '/detail': (context) => DetailScreen(),
      },
    );
  }
}

class HomeScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Home Screen"),
      ),

      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [

            ElevatedButton(
              child: Text("Go to Second Screen (push)"),
              onPressed: () {
                Navigator.push(
                  context,
                  MaterialPageRoute(
                    builder: (context) => SecondScreen(),
                  ),
                );
              },
            ),

            SizedBox(height: 20),
```

```
            ElevatedButton(
              child: Text("Go to Second Screen (pushNamed)"),
              onPressed: () {
                Navigator.pushNamed(context, '/second');
              },
            ),

            SizedBox(height: 20),

            ElevatedButton(
              child: Text("Send Data to Detail Screen"),
              onPressed: () {
                Navigator.pushNamed(
                  context,
                  '/detail',
                  arguments: "Hello from Home Screen",
                );
              },
            ),
          ],
        ),
      ),
    );
  }
}

class SecondScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Second Screen"),
      ),

      body: Center(
        child: ElevatedButton(
          child: Text("Go Back"),
          onPressed: () {
            Navigator.pop(context);
```

```
          },
        ),
      ),
    );
  }
}

class DetailScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {

    final message =
        ModalRoute.of(context)!.settings.arguments as String;

    return Scaffold(
      appBar: AppBar(
        title: Text("Detail Screen"),
      ),

      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [

            Text(
              message,
              style: TextStyle(fontSize: 20),
            ),

            SizedBox(height: 20),

            ElevatedButton(
              child: Text("Back"),
              onPressed: () {
                Navigator.pop(context);
              },
            )
          ],
        ),
```

```
        ),
      );
    }
  }
```